Norwegian Centre on AI for Decisions

Probabilistic Modelling with NumPyro

Workshop

Helge Langseth (NTNU), Signe Riemer-Sørensen (SINTEF), Alexander Johannes Stasik (SINTEF & NMBU)

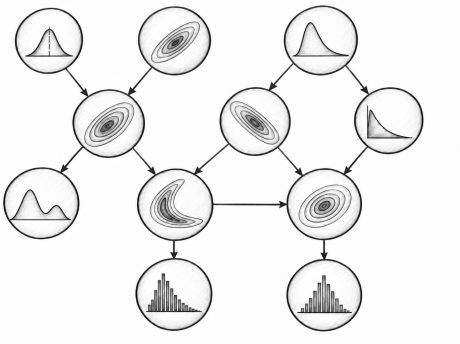
October 14, 2026

What is Probabilistic Modelling?

- Describing systems using probability distributions
- Accounting for uncertainty in input data
- Accounting for uncertainty in model parameters and stochastic processes

What is Probabilistic Modelling?

- Describing systems using probability distributions
- Accounting for uncertainty in input data
- Accounting for uncertainty in model parameters and stochastic processes



Why Probabilistic Modelling?

Why Probabilistic Modelling?

A prediction without uncertainty can be misleading.

Why Probabilistic Modelling?

A prediction without uncertainty can be misleading.

Example: Decisions with uncertainty

The most likely amount of rainfall tomorrow in Bergen is 0 mm. Would you bring an umbrella?

Why Probabilistic Modelling?

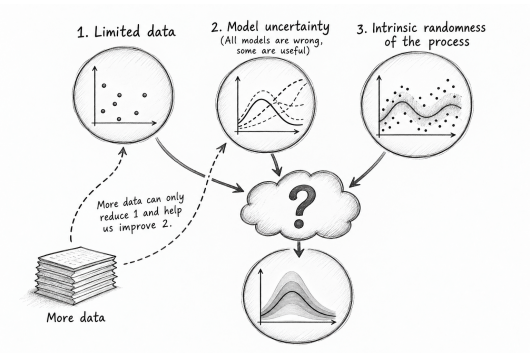
Uncertainty is not your enemy, but a key part of the model.

How can we describe uncertainty?

- Using categories (chance of project failure is low, chance of some deviation is high)
- Assigning probabilities to specific outcomes (chance of a total power failure is 0.1%, chance of one to three local blackouts is 3%)
- Hypothesis testing and p-values
- A full probability distribution (see the coin-toss example)

Sources of uncertainty

1. Limited data
2. Model uncertainty (All models are wrong, some are useful)
3. Intrinsic randomness of the process



Frequentist vs Bayesian interpretation

- Two approaches to statistical inference
- Fundamentally different interpretations of uncertainty
- Different assumptions and methodological challenges
- Can lead to similar results with large datasets
- Important differences with finite datasets



The common principle

The likelihood function

$$L(\boldsymbol{x}|\theta)$$

describes how likely the observed data are under different model parameters. Typically, our goal is to infer the model parameters θ.

The common principle

The likelihood function

$$L(x|\theta)$$

describes how likely the observed data are under different model parameters. Typically, our goal is to infer the model parameters θ.

$$L(x|p) = p^x(1-p)^{1-x}$$

and

$$L(\mathbf{x}|p) = \prod_{i=1}^{n} p^{x_i}(1-p)^{1-x_i}$$

with **x** as an observation sequence



The Frequentist approach

Given the observation, we want an estimate $\hat{\theta}$ for the true parameters θ_{true}.

The Frequentist approach

Given the observation, we want an estimate $\hat{\theta}$ for the true parameters θ_{true}. A common approach is maximum likelihood estimation.

Maximum likelihood

$$\hat{\theta} = \arg\max_{\theta} L(\theta; x)$$

The Frequentist approach

Given the observation, we want an estimate $\hat{\theta}$ for the true parameters θ_{true}. A common approach is maximum likelihood estimation.

Maximum likelihood

$$\hat{\theta} = \arg\max_{\theta} L(\theta; x)$$

We seek the parameter estimate that maximises the likelihood of the observed data.

The Frequentist approach

Given the observation, we want an estimate $\hat{\theta}$ for the true parameters θ_{true}. A common approach is maximum likelihood estimation.

Maximum likelihood

$$\hat{\theta} = \arg\max_{\theta} L(\theta; x)$$

We seek the parameter estimate that maximises the likelihood of the observed data. In our example this is

$$\hat{p}_{\text{MLE}} = \arg\max_{p \in [0,1]} L(p; \mathbf{x})$$

The Frequentist approach

1. Acquire data
2. Build a model (likelihood)
3. Perform optimisation w.r.t. model parameters (maximum likelihood)
4. Quantify estimation uncertainty using standard errors, confidence intervals, or sampling distributions

The Frequentist approach

1. Acquire data
2. Build a model (likelihood)
3. Perform optimisation w.r.t. model parameters (maximum likelihood)
4. Quantify estimation uncertainty using standard errors, confidence intervals, or sampling distributions

Frequentist interpretation

In the frequentist framework, the model parameters are treated as fixed but unknown. Uncertainty is quantified through the sampling distribution of estimators across hypothetical repetitions of the experiment.

The Bayesian approach

$$p(\theta \mid x) =$$

The Bayesian approach

$$p(\theta \mid x) = \frac{p(x \mid \theta)p(\theta)}{p(x)} = \frac{\mathcal{L}(x \mid \theta)\pi(\theta)}{\int p(x, \theta)\, \mathrm{d}\theta}$$

The Bayesian approach

$$p(\theta \mid x) = \frac{p(x \mid \theta)p(\theta)}{p(x)} = \frac{\mathcal{L}(x \mid \theta)\pi(\theta)}{\int p(x, \theta)\, \mathrm{d}\theta}$$

with

- $p(\theta \mid x)$: posterior, our knowledge after seeing the data
- $\mathcal{L}(x \mid \theta)$: likelihood, the model for the data given θ
- $\pi(\theta)$: prior, our beliefs before seeing the data
- x: observed data
- θ: model parameters

A short derivation

The product rule for probabilities gives

$$p(x, y) = p(x \mid y)p(y) = p(y \mid x)p(x).$$

A short derivation

The product rule for probabilities gives

$$p(x, y) = p(x \mid y)p(y) = p(y \mid x)p(x).$$

Rearranging,

$$p(y \mid x) = \frac{p(x \mid y)p(y)}{p(x)}.$$

A short derivation

The product rule for probabilities gives

$$p(x, y) = p(x \mid y)p(y) = p(y \mid x)p(x).$$

Rearranging,

$$p(y \mid x) = \frac{p(x \mid y)p(y)}{p(x)}.$$

Setting $y = \theta$ gives Bayes' rule.

Doing Bayesian inference

Remember

$$p(\theta \mid x) = \frac{\mathcal{L}(x \mid \theta)\pi(\theta)}{p(x)}.$$

Doing Bayesian inference

Remember

$$p(\theta \mid x) = \frac{\mathcal{L}(x \mid \theta)\pi(\theta)}{p(x)}.$$

The key problem is

$$p(x) = \int p(x, \theta)\, \mathrm{d}\theta.$$

We can easily compute

$$p(\theta \mid x) \propto \mathcal{L}(x \mid \theta)\pi(\theta),$$

but the posterior must be normalised.

Doing Bayesian inference

Remember

$$p(\theta \mid x) = \frac{\mathcal{L}(x \mid \theta)\pi(\theta)}{p(x)}.$$

The key problem is

$$p(x) = \int p(x, \theta)\, \mathrm{d}\theta.$$

We can easily compute

$$p(\theta \mid x) \propto \mathcal{L}(x \mid \theta)\pi(\theta),$$

but the posterior must be normalised. For some prior–likelihood combinations this can be done analytically, but in general it cannot.

The Bayesian approach

1. Specify a prior distribution
2. Acquire data
3. Build a model (likelihood)
4. Apply Bayes' theorem

The Bayesian approach

1. Specify a prior distribution
2. Acquire data
3. Build a model (likelihood)
4. Apply Bayes' theorem

In the Bayesian framework, model parameters are treated as uncertain quantities. We use probability distributions to express our knowledge about them and update these distributions as new data become available.

Comparison to frequentist statistics

Frequentist inference often leads to an optimisation problem, for example maximum likelihood.

Comparison to frequentist statistics

Frequentist inference often leads to an optimisation problem, for example maximum likelihood. Bayesian inference often leads to an integration problem.

Frequentist: often optimisation. Bayesian: often integration.

For the rest of this workshop, we will focus on Bayesian inference.

Coin Toss Example

See notebook o

Conjugate Priors in Bayesian Inference

For certain prior–likelihood combinations, the product

$$\mathcal{L}(\boldsymbol{x} \mid \theta)\, \pi(\theta)$$

yields a posterior in the same distribution family as the prior. These are called conjugate priors and allow us to derive the posterior analytically.

Conjugate Priors in Bayesian Inference

For certain prior–likelihood combinations, the product

$$\mathcal{L}(\boldsymbol{x} \mid \theta)\, \pi(\theta)$$

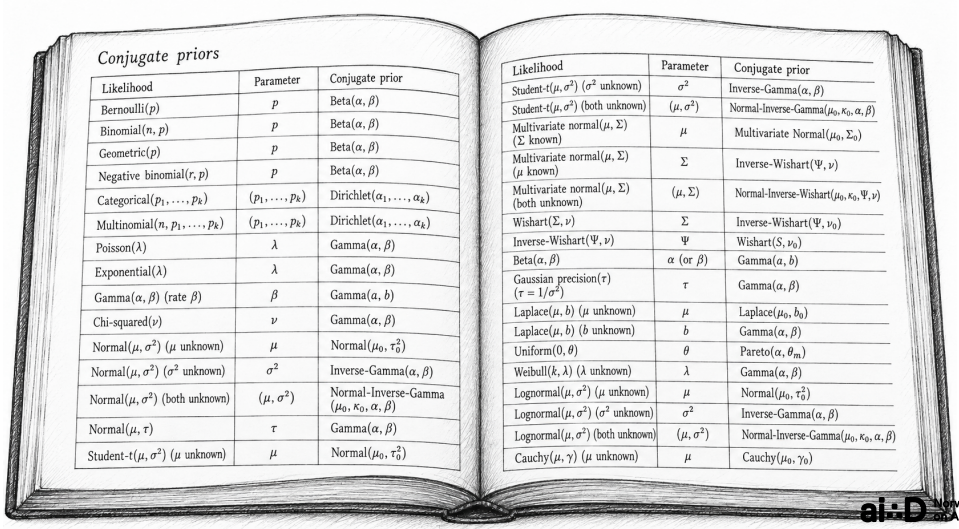
yields a posterior in the same distribution family as the prior. These are called conjugate priors and allow us to derive the posterior analytically.

Bernoulli experiment

The Beta distribution is a conjugate prior for the Bernoulli likelihood.
If the prior is $\mathrm{Beta}(\alpha, \beta)$, the posterior is

$$\mathrm{Beta}(\alpha + n_{\text{successes}},\ \beta + n_{\text{trials}} - n_{\text{successes}}).$$

Conjugate Priors in Bayesian Inference



Conjugate priors

Likelihood	Parameter	Conjugate prior
Bernoulli(p)	p	Beta(α, β)
Binomial(n, p)	p	Beta(α, β)
Geometric(p)	p	Beta(α, β)
Negative binomial(r, p)	p	Beta(α, β)
Categorical($p_1, \ldots, p_k$)	$(p_1, \ldots, p_k)$	Dirichlet($\alpha_1, \ldots, \alpha_k$)
Multinomial($n, p_1, \ldots, p_k$)	$(p_1, \ldots, p_k)$	Dirichlet($\alpha_1, \ldots, \alpha_k$)
Poisson(λ)	λ	Gamma(α, β)
Exponential(λ)	λ	Gamma(α, β)
Gamma(α, β) (rate β)	β	Gamma(a, b)
Chi-squared(ν)	ν	Gamma(α, β)
Normal(μ, σ^2) (μ unknown)	μ	Normal(μ_0, τ_0^2)
Normal(μ, σ^2) (σ^2 unknown)	σ^2	Inverse-Gamma(α, β)
Normal(μ, σ^2) (both unknown)	(μ, σ^2)	Normal-Inverse-Gamma ($\mu_0, \kappa_0, \alpha, \beta$)
Normal(μ, τ)	τ	Gamma(α, β)
Student-t(μ, σ^2) (μ unknown)	μ	Normal(μ_0, τ_0^2)

Likelihood	Parameter	Conjugate prior
Student-t(μ, σ^2) (σ^2 unknown)	σ^2	Inverse-Gamma(α, β)
Student-t(μ, σ^2) (both unknown)	(μ, σ^2)	Normal-Inverse-Gamma($\mu_0, \kappa_0, \alpha, \beta$)
Multivariate normal(μ, Σ) (Σ known)	μ	Multivariate Normal(μ_0, Σ_0)
Multivariate normal(μ, Σ) (μ known)	Σ	Inverse-Wishart(Ψ, ν)
Multivariate normal(μ, Σ) (both unknown)	(μ, Σ)	Normal-Inverse-Wishart($\mu_0, \kappa_0, \Psi, \nu$)
Wishart(Σ, ν)	Σ	Inverse-Wishart(Ψ, ν_0)
Inverse-Wishart(Ψ, ν)	Ψ	Wishart(S, ν_0)
Beta(α, β)	α (or β)	Gamma(a, b)
Gaussian precision(τ) ($\tau = 1/\sigma^2$)	τ	Gamma(α, β)
Laplace(μ, b) (μ unknown)	μ	Laplace(μ_0, b_0)
Laplace(μ, b) (b unknown)	b	Gamma(α, β)
Uniform($0, \theta$)	θ	Pareto(α, θ_m)
Weibull(k, λ) (λ unknown)	λ	Gamma(α, β)
Lognormal(μ, σ^2) (μ unknown)	μ	Normal(μ_0, τ_0^2)
Lognormal(μ, σ^2) (σ^2 unknown)	σ^2	Inverse-Gamma(α, β)
Lognormal(μ, σ^2) (both unknown)	(μ, σ^2)	Normal-Inverse-Gamma($\mu_0, \kappa_0, \alpha, \beta$)
Cauchy(μ, γ) (μ unknown)	μ	Cauchy(μ_0, γ_0)

The inference problem

Recall: The posterior of the parameters θ given the data x is

$$p(\theta \mid x) = \frac{\mathcal{L}(x \mid \theta)\,\pi(\theta)}{p(x)}, \qquad p(x) = \int \mathcal{L}(x \mid \theta)\,\pi(\theta)\,d\theta \quad \text{(intractable in general).}$$

What we can evaluate

Only the unnormalised density $\tilde{p}(\theta) = \mathcal{L}(x \mid \theta)\,\pi(\theta) \propto p(\theta \mid x)$, and (sometimes) its gradient $\nabla_\theta \log \tilde{p}(\theta)$.

What we typically need are expectations $\mathbb{E}_{p(\theta|x)}[f(\theta)]$ under the posterior, for example:

$$\underbrace{f(\theta) = \theta}_{\text{posterior mean}} \qquad \underbrace{f(\theta) = \mathbf{1}[\theta \in A]}_{\Pr(\theta \in A \mid x)} \qquad \underbrace{f(\theta) = p(x^* \mid \theta)}_{p(x^* \mid x),\ \text{the predictive density}}$$

(A: A set of parameter values; x^*: A new observation.)

Monte Carlo: Replace the integral by an average

If we had *independent* samples $\theta^{(1)}, \ldots, \theta^{(N)} \sim p(\theta \mid x)$ (below: p for short), then

$$\mathbb{E}_p[f(\theta)] \;\approx\; \hat{\mu}_N = \frac{1}{N}\sum_{s=1}^{N} f\big(\theta^{(s)}\big).$$

- Unbiased, and $\mathrm{Var}[\hat{\mu}_N] = \mathrm{Var}_p[f]/N$: The error shrinks as $1/\sqrt{N}$, a rate independent of the dimension (for independent draws).
- So sampling turns the inference problem into: How do we draw from p when we only know $\tilde{p} \propto p$?

Sampling is approximate inference: The samples provide the approximation, and the error can be made as small as we like, given enough computation.

A simple approach: Importance sampling

We have no direct sampler for a general unnormalised $\tilde{p}(\theta)$. Instead, draw $\theta^{(1)}, \ldots, \theta^{(N)}$ from an easy proposal distribution q (e.g. a Gaussian) and correct with importance weights

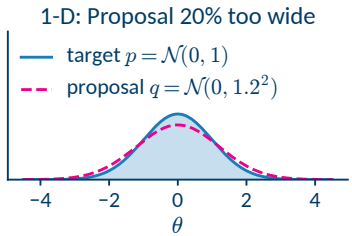
$$w^{(s)} = \frac{\tilde{p}(\theta^{(s)})}{q(\theta^{(s)})}, \qquad \mathbb{E}_p[f] \approx \frac{\sum_s w^{(s)} f(\theta^{(s)})}{\sum_s w^{(s)}}.$$

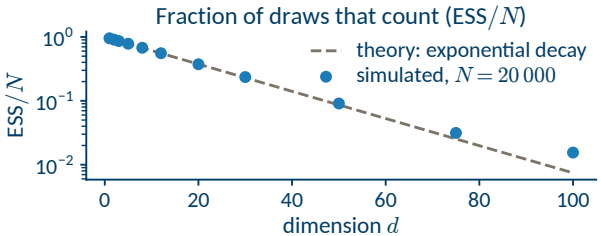
- Works with the unnormalised $\tilde{p}$, and is consistent for any q that covers p.
- If q differs from p, a few weights are large and many tiny: Fewer than N draws really contribute.
- The effective sample size (ESS) N_{eff} counts them:

$$N_{\text{eff}} = \frac{(\sum_s w^{(s)})^2}{\sum_s (w^{(s)})^2}.$$

  Equal weights give $N_{\text{eff}} = N$; one dominating sample gives $N_{\text{eff}} \approx 1$. Roughly, N weighted draws are *worth* N_{eff} independent draws from p.

Importance sampling breaks down in high dimension





Target $p = \mathcal{N}(0, I_d)$, proposal $q = \mathcal{N}(0, \tau^2 I_d)$, $\tau = 1.2$. **Left:** $d = 1$. **Right:** ESS/N against d.

Main problem with importance sampling

Note how the mismatch per dimension compounds so that ESS/N declines exponentially in d. We would prefer a sampling scheme (still only using the unnormalised $\tilde{p}$), for which ESS/N degrades more slowly.

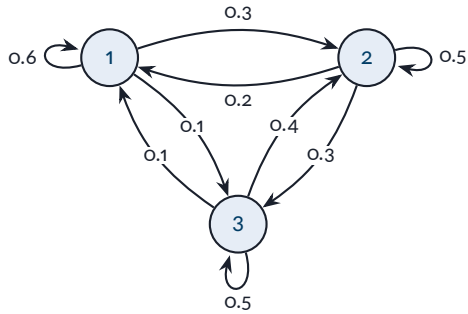
A small side-step: Markov chains

A Markov chain is a sequence of states $\theta^{(0)}, \theta^{(1)}, \ldots$ (here: The parameter θ we want to sample) with the Markov property, where $\theta^{(0:t)}$ is the history up to time t:

$$p(\theta^{(t+1)} \mid \theta^{(0:t)}) = p(\theta^{(t+1)} \mid \theta^{(t)}).$$

On a finite state space the dynamics are described by a transition matrix $P_{ij} = p(\theta^{(t+1)} = j \mid \theta^{(t)} = i)$; the distribution p_t of $\theta^{(t)}$ (a row vector) evolves as

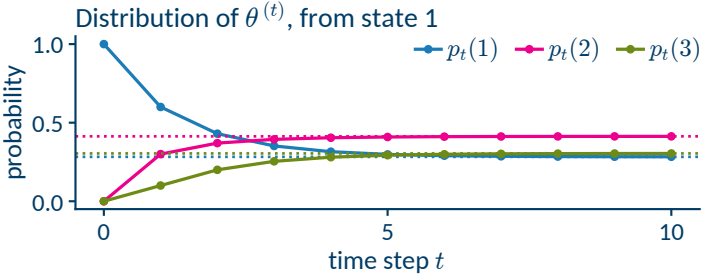
$$p_{t+1} = p_t P, \qquad p_t = p_0 P^t.$$



$$P = \begin{pmatrix} 0.6 & 0.3 & 0.1 \\ 0.2 & 0.5 & 0.3 \\ 0.1 & 0.4 & 0.5 \end{pmatrix}$$

Limiting distribution

Convergence theorem (finite state space). If the chain is irreducible (every state reachable from every other) and aperiodic (no deterministic cycling), there is a unique *stationary* distribution $p_\star = p_\star P$, and $p_t \to p_\star$ from *any* p_0, geometrically at rate $|\lambda_2|$ (second-largest eigenvalue modulus).



$p_\star \approx (0.283, 0.413, 0.304)$, shown as dotted lines.

Clever idea: Construct a chain with the “right” limiting distribution

Instead of analysing a given chain, defined by a transition density $k(\theta' \mid \theta)$ (sums become integrals when θ is continuous), we design one whose stationary distribution $p_\star$ is the target p.

Detailed balance (reversibility)

$$p(\theta)\, k(\theta' \mid \theta) = p(\theta')\, k(\theta \mid \theta') \quad \text{for all } \theta, \theta'.$$

Equilibrium: The probability flow $\theta \to \theta'$ equals the flow $\theta' \to \theta$: No net flow, so p stays put.

Detailed balance implies (sufficient not necessary) that p is stationary, i.e. unchanged by one step. With irreducibility and aperiodicity the chain also *converges* to p.

Clever idea: Construct a chain with the “right” limiting distribution

Instead of analysing a given chain, defined by a transition density $k(\theta' \mid \theta)$ (sums become integrals when θ is continuous), we design one whose stationary distribution $p_\star$ is the target p.

Detailed balance (reversibility)

$$p(\theta)\, k(\theta' \mid \theta) = p(\theta')\, k(\theta \mid \theta') \quad \text{for all } \theta, \theta'.$$

Equilibrium: The probability flow $\theta \to \theta'$ equals the flow $\theta' \to \theta$: No net flow, so p stays put.

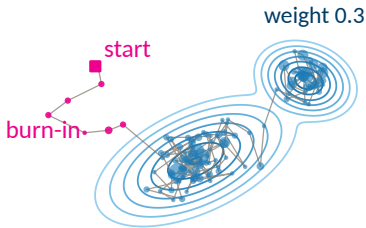
Detailed balance implies (sufficient not necessary) that p is stationary, i.e. unchanged by one step. With irreducibility and aperiodicity the chain also *converges* to p.

$\Longrightarrow$ Our task is to find $k(\theta' \mid \theta)$ such that detailed balance holds.

MCMC: Markov chain Monte Carlo

Markov chain Monte Carlo (MCMC): Build a Markov chain $\theta^{(1)}, \theta^{(2)}, \ldots$ whose long-run distribution is $p(\theta \mid x)$. It only needs $\tilde{p}$.

- Samples are dependent, yet the ergodic theorem still gives $\frac{1}{N}\sum_s f(\theta^{(s)}) \to \mathbb{E}_p[f]$ almost surely.
- The price is autocorrelation. With ρ_t the lag-t autocorrelation of $f(\theta^{(s)})$ in the stationary chain, the effective sample size (ESS) is $N_{\text{eff}} = N \big/ \left(1 + 2\sum_{t \geq 1} \rho_t\right)$: N dependent draws are worth N_{eff} independent ones.
- Early draws, before the chain has found its long-run behaviour, are discarded as burn-in (*warm-up*).



Example run. Dot size ∼ No. iterations the chain stayed at that position. Jumps between modes are rare.

The whole game: High ESS *per unit of computation*, counted in target evaluations.

The Metropolis–Hastings (MH) algorithm

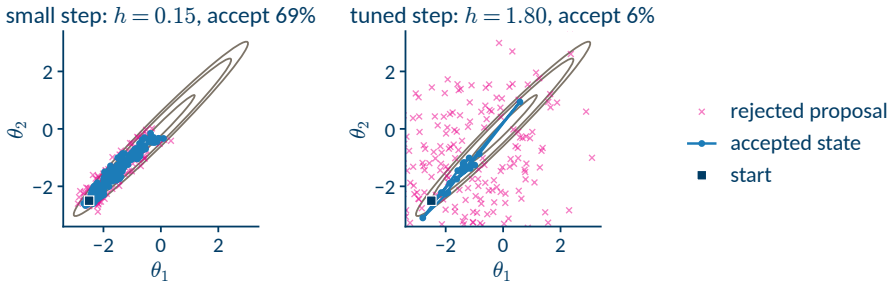
1: **input:** $\tilde{p}$, proposal $q(\theta' \mid \theta)$, start $\theta^{(0)}$
2: **for** $s = 1, \ldots, N$ **do**
3: draw a proposal $\theta' \sim q(\cdot \mid \theta^{(s-1)})$
4: $\alpha \leftarrow \min\left\{1, \dfrac{\tilde{p}(\theta')\, q(\theta^{(s-1)} \mid \theta')}{\tilde{p}(\theta^{(s-1)})\, q(\theta' \mid \theta^{(s-1)})}\right\}$
5: draw $u \sim \mathrm{Unif}(0, 1)$
6: $\theta^{(s)} \leftarrow \theta'$ if $u < \alpha$, else $\theta^{(s)} \leftarrow \theta^{(s-1)}$
7: **end for**

- Detailed balance holds for any proposal q, so p is stationary. For $\theta' \neq \theta$ the chain's kernel is $k(\theta' \mid \theta) = q(\theta' \mid \theta)\, \alpha(\theta, \theta')$, with $\alpha(\theta, \theta')$ the acceptance probability of line 4 (for $\theta = \theta^{(s-1)}$). This choice of α makes $p(\theta)k(\theta' \mid \theta)$ symmetric.
- A rejection repeats the current state, and is treated as any other sample.
- Normalization constant for $p(\theta)$ cancels in the ratio $\alpha(\theta, \theta')$, so only $\tilde{p}$ is needed.

Random-walk Metropolis (RWM): $q(\theta' \mid \theta) = \mathcal{N}(\theta' \mid \mu = \theta, \Sigma = h^2 I)$, with h denoted "step size", is symmetric, so $\alpha = \min\{1, \tilde{p}(\theta')/\tilde{p}(\theta)\}$. Moves to higher density always accepted, to lower density with probability equal to density ratio.

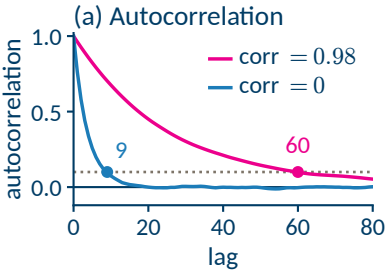
Demo: Random-walk Metropolis on a correlated Gaussian

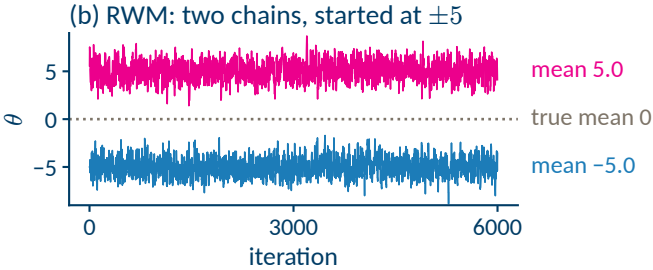
Running example: $\theta = (\theta_1, \theta_2) \sim \mathcal{N}(0, \Sigma)$, unit variances, correlation $\mathrm{corr}(\theta_1, \theta_2) = 0.98$; standard deviations along the ellipse axes $\sigma_{\max} = 1.41$ and $\sigma_{\min} = 0.14$. Steps must be small for the narrow direction, yet the chain must travel the wide one. 250 iterations from $(-2.5, -2.5)$.



Left: Slow mixing despite a high acceptance. **Right:** Best step size (w.r.t. ESS) has a low acceptance rate.

Weaknesses in action: Random-walk Metropolis



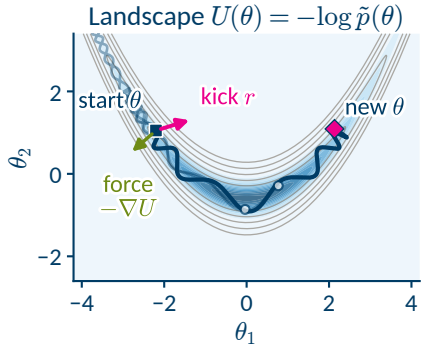


(a) $d = 2$. RWM (tuned step) on a 2-D Gaussian with independent coordinates (corr $= 0$) and on our running example (corr $= 0.98$). The autocorrelation falls below 0.1 (dotted line) at the lags marked.

(b) Mixture of two 1-D Gaussians, $\mathcal{N}(-5, 1)$ and $\mathcal{N}(5, 1)$. We run two chains, one per mode. Each Markov chain stays put for 6 000 iterations and look stationary/converged. We need to run several chains with dispersed starts to reveal the problem.

Hamiltonian Monte Carlo (HMC): The idea

- ESS$/N$ can be very low for random walks. Let's use $\nabla \log \tilde{p}$ for long, informed moves! This needs continuous θ and gradients. Imagine a puck on a frictionless landscape $U(\theta) = -\log \tilde{p}(\theta)$ (so $p \propto e^{-U}$). It gets a random kick r and slides along the surface; the force $-\nabla U$ bends its path. We read off the puck's position after time T.
- Formally, $r \sim \mathcal{N}(0, M)$ with *mass matrix* M (velocity $M^{-1}r$); for now, $M = I$. The pair (θ, r) has density $\propto e^{-H}$ with $H = U(\theta) + \frac{1}{2} r^\top M^{-1} r$.
- The θ-marginal is the target: Sample (θ, r), keep θ, draw a fresh r every iteration.
- Hamilton's equations $\dot{\theta} = M^{-1}r, \dot{r} = -\nabla U(\theta)$ conserve H and phase-space volume, hence leave e^{-H} invariant: Exact dynamics are always accepted. Integration is approximate (next slide); a Metropolis step corrects the error.



Thick line: Trajectory after one kick. The puck is at the square after time T ($= \epsilon L$, next slide). This is the new θ. **Faint:** Other kicks with ○ as end points. All paths run for the same time T.

Leapfrog integration and the HMC algorithm

One leapfrog step

In general the dynamics cannot be solved exactly, so we discretise with the leapfrog integrator with step size ϵ:

- Half kick: $r_{t+\epsilon/2} = r_t - \frac{\epsilon}{2}\nabla U(\theta_t)$
- Full drift: $\theta_{t+\epsilon} = \theta_t + \epsilon\, r_{t+\epsilon/2}$
- Half kick: $r_{t+\epsilon} = r_{t+\epsilon/2} - \frac{\epsilon}{2}\nabla U(\theta_{t+\epsilon})$

Accept or reject

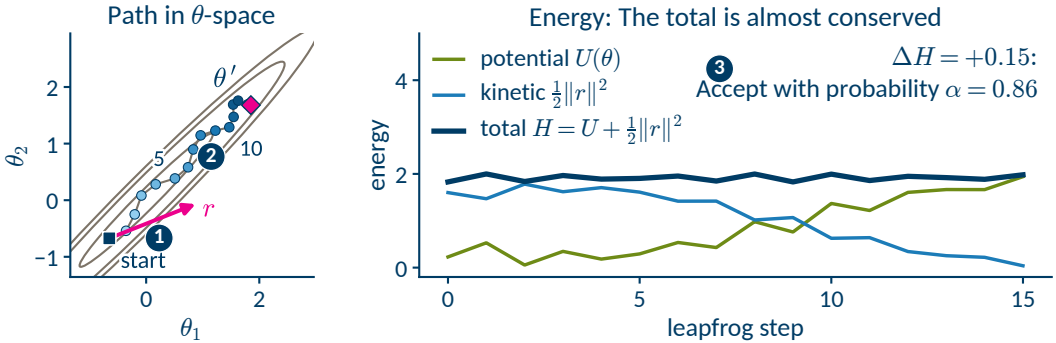
After L leapfrog steps, from (θ, r) to (θ', r'), accept θ' with probability

$$\alpha = \min\left\{1, e^{-\Delta H}\right\}, \quad \Delta H = H(\theta', r') - H(\theta, r);$$

otherwise keep θ.

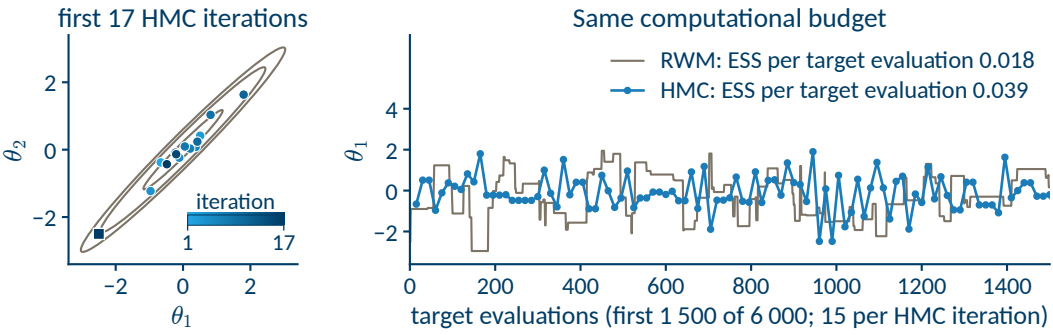
Hyperparameters: ϵ (step size) and L (number of steps; trajectory length $T = \epsilon L$).
Cost unit: One target evaluation (with gradient) per leapfrog step.

Demo: One HMC iteration – step by step



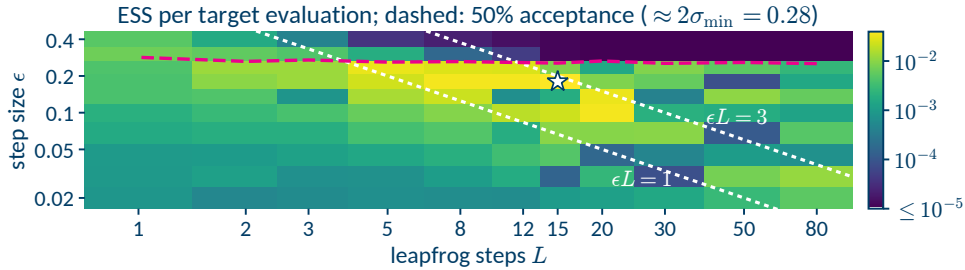
1. Kick: Draw $r \sim \mathcal{N}(0, I)$ ($\epsilon = 0.18$, $L = 15$; an accepted iteration with a long jump, picked for display). **2. Roll:** The leapfrog steps use only $\nabla \log \tilde{p}$. **3. Accept or reject** with probability $\min\{1, e^{-\Delta H}\}$. **Result:** A long jump, still with high acceptance probability, which a random-walk proposal cannot deliver.

Demo: HMC on the same correlated Gaussian



Left: The first 17 HMC iterations ($\epsilon = 0.18$, $L = 15$; 255 target evaluations) from the starting point (square) of the random-walk demo. **Right:** θ_1 for the same budget of target evaluations; RWM uses its tuned step $h = 1.8$. With a well-chosen (ϵ, L) HMC beats random-walk Metropolis by a factor of 2.2 in ESS per target evaluation, a modest gain in 2-D.

Step size and trajectory length: Why they are hard to choose



Each cell: Median ESS per target evaluation over five HMC runs of 6 000 iterations; star: Best cell ($\epsilon = 0.18$, $L = 15$).

1. Two scales at once: ϵ is capped by the narrowest direction ($\epsilon < 2\sigma_{\min} \approx 0.28$), but $T = \epsilon L$ must cover the widest.
2. No smooth landscape: Acceptance only flags ϵ too large; ESS per target evaluation is high for $\epsilon L \approx 1$–3, but irregular in L (resonances, not noise) and known only *after* sampling.
3. Model- and region-dependent: Optimal values change with the model, the data, and the region.
4. A third hyperparameter: The mass matrix M rescales and rotates the problem.

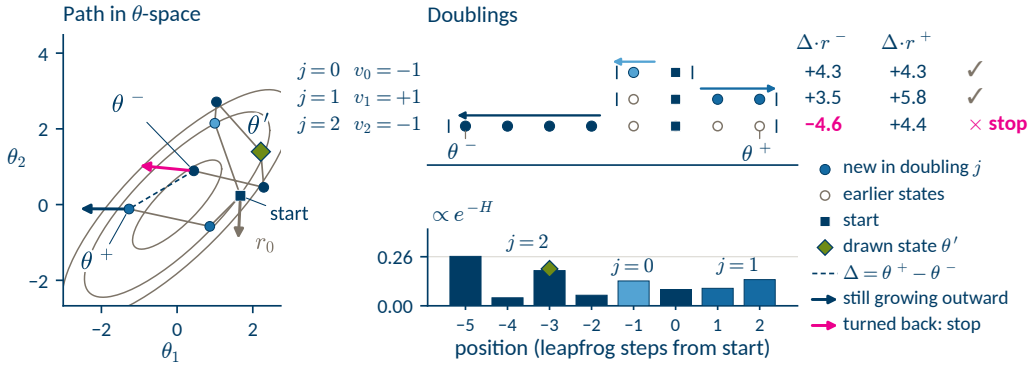
NUTS: Choose the trajectory length automatically

The No-U-Turn sampler (NUTS) is HMC with an automatic choice of L. One transition:

1. Draw the initial kick $r_0 \sim \mathcal{N}(0, M)$; the trajectory starts as the single state (θ, r_0).
2. For $j = 0, 1, 2, \ldots$:
   - Doubling: Draw a direction $v_j \in \{-1, +1\}$ and add 2^j leapfrog steps (step size $v_j\epsilon$) at that end of the trajectory, continuing from its end state $(\theta^{\pm}, r^{\pm})$.
   - Stop: When either end would, by growing further, move toward the other end: $(\theta^+ - \theta^-) \cdot r^+ < 0$ or $(\theta^+ - \theta^-) \cdot r^- < 0$.
3. Select θ' from the states of the final trajectory with probability $\propto e^{-H}$. Keep θ', discard r and the rest of the trajectory.

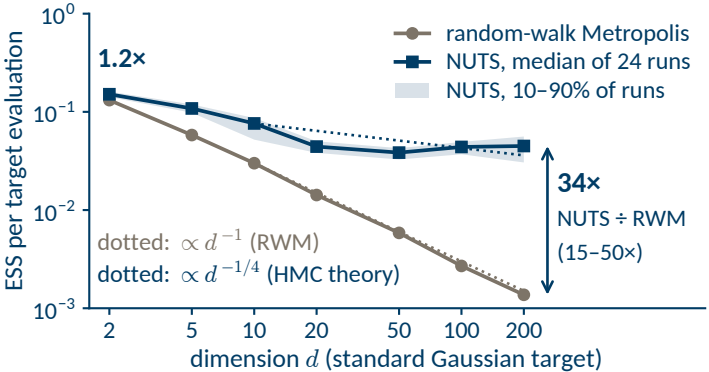
ϵ is tuned during warm-up so that the average acceptance probability reaches a target δ (default 0.8). Warm-up can also learn M.

Demo: One NUTS transition – step by step



Toy target (corr $= 0.8, \epsilon = 0.6$; the ridge of our running example is too thin to show the geometry). **Left:** $\Delta = \theta^+ - \theta^-$ (dashed). **Top right:** At $j = 2, \Delta \cdot r^- < 0$ (magenta): growing further back would head toward θ^+, so growth stops at 8 states. **Bottom:** θ' is drawn among them with probability $\propto e^{-H}$.

Where gradients pay off: Higher dimensions



Bold: Ratio of ESS per target evaluation, NUTS ÷ RWM (median of 24 runs); NUTS cost excludes warm-up.

Takeaway: The real argument for NUTS is in high dimensions; in 2-D the gain over RWM is modest.

Alternative: Variational inference (VI)

Optimizing instead of sampling: Choose a simple family $q_\phi(\theta)$ and make it as close to the posterior as possible,

$$\min_\phi \ \mathrm{KL}\big(q_\phi \,\big\|\, p(\cdot \mid x)\big) \iff \max_\phi \ \mathrm{ELBO}(\phi), \quad \mathrm{ELBO}(\phi) = \mathbb{E}_{q_\phi}\big[\log \tilde{p}(\theta) - \log q_\phi(\theta)\big].$$

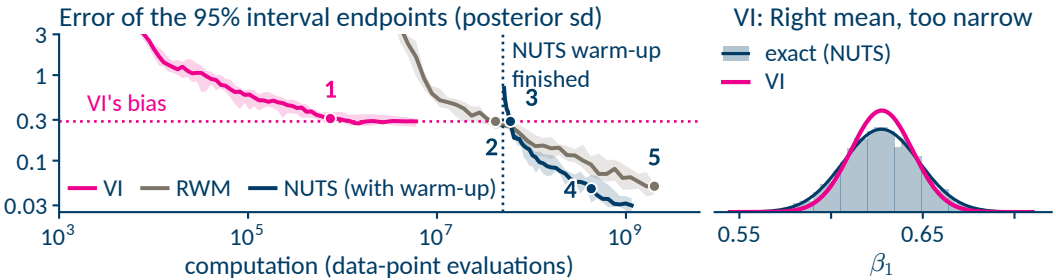
- Typical: Mean-field Gaussian, $q_\phi = \prod_j \mathcal{N}(\theta_j; \mu_j, \sigma_j^2)$.
- Stochastic VI (SVI): ELBO gradients from automatic differentiation on minibatches of data.
- VI only needs $\tilde{p}$ and $\nabla \log \tilde{p}$.

Pros and cons of mean-field VI compared with MCMC

\+ Fast: Optimisation, scales to big data.

– Biased: Limited by the family. Difficult to measure bias. Mean-field ignores correlations.

VI against MCMC: Faster, but biased



Example: Bayesian logistic regression, $n = 20\,000$, $d = 10$. An SVI step uses a minibatch of 200 data points, a NUTS or RWM step all n. Error: Mean absolute error of the 95% interval endpoints, in posterior-sd units, against a long NUTS run.

- VI: Within 10% of its final error at $\approx 7 \times 10^5$ data-point evaluations (**1**), intervals 15% too narrow.
- Passing VI: RWM at 4×10^7 (**2**), without separate warm-up; NUTS at 6×10^7 (**3**), of which warm-up is 5×10^7.
- Reaching error 0.05: NUTS at 4×10^8 (**4**), RWM only at 2×10^9 (**5**); NUTS keeps improving.

Take-home messages (approximate inference)

1. MCMC samples from $p(\theta \mid x)$ using only $\tilde{p}$. Detailed balance lets us design the chain, and Metropolis–Hastings is the generic recipe.
2. Random-walk proposals diffuse: The cost grows with dimension and correlation. HMC uses gradients (so continuous parameters) for long moves, but ϵ, L and M are hard to set.
3. NUTS adapts most of that away (L automatically, ϵ and M in warm-up), and its edge is high dimension (about $34\times$ over RWM at $d = 200$). There are still remaining issues, though! (Signe will show you.)
4. Variational inference trades exactness for speed: Fast, but biased (mean-field: Too narrow), and no budget removes that. MCMC is slower but eventually exact.

Probabilistic Programming

Doing Bayesian inference by hand quickly becomes difficult.

Probabilistic Programming

Doing Bayesian inference by hand quickly becomes difficult. **Probabilistic programming frameworks** can automate much of this process:

- Allow us to describe complex probabilistic models
- Separate model specification from inference
- Support automated inference for unknown quantities
- We will use `NumPyro` based on `JAX`

Probabilistic Programming

Doing Bayesian inference by hand quickly becomes difficult. **Probabilistic programming frameworks** can automate much of this process:

- Allow us to describe complex probabilistic models
- Separate model specification from inference
- Support automated inference for unknown quantities
- We will use `NumPyro` based on `JAX`
- Similar framework exists, for example `turning` in `julia`

NumPyro in a Nutshell

- A probabilistic programming framework for Python
- Built on top of `JAX`
- Supports automatic differentiation and JIT compilation
- Supports efficient computation on CPUs, GPUs, and TPUs
- Particularly well suited for gradient-based inference such as NUTS

NumPyro in a Nutshell

- A probabilistic programming framework for Python
- Built on top of `JAX`
- Supports automatic differentiation and JIT compilation
- Supports efficient computation on CPUs, GPUs, and TPUs
- Particularly well suited for gradient-based inference such as NUTS

Why JAX?

Modern MCMC methods need gradients of the log posterior. JAX can compute these gradients automatically.

Anatomy of a `numpyro` model: Defining a model

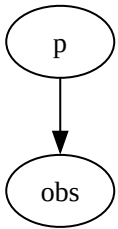
```
import numpyro
import numpyro.distributions as dist

def model(data=None):
    p = numpyro.sample("p", dist.Beta(1., 1.))
    numpyro.sample("obs", dist.Bernoulli(p), obs=data)
```

A simple model with a Beta prior and Bernoulli likelihood.

Anatomy of a `numpyro` model: Defining a model

```
numpyro.render_model(model, render_distributions=True)
```



p ~ Beta
obs ~ BernoulliProbs

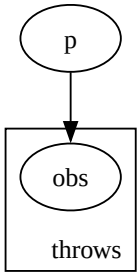
Anatomy of a `numpyro` model: Defining a model

```
def model(data=None, n=10):
    p = numpyro.sample("p", dist.Beta(1., 1.))
    with numpyro.plate("throws", n):
        numpyro.sample("obs", dist.Bernoulli(p), obs=data)
```

A simple model with a Beta prior and multiple iid Bernoulli observations.

Anatomy of a `numpyro` model: Defining a model

```
numpyro.render_model(model, render_distributions=True)
```



p ~ Beta
obs ~ BernoulliProbs

Anatomy of a `numpyro` model: Doing inference

Create a small toy dataset and do inference:

Anatomy of a `numpyro` model: Doing inference

Create a small toy dataset and do inference:

```
import jax
import jax.numpy as jnp
from numpyro.infer import MCMC, NUTS

data = jnp.array([0, 1, 0, 1, 1, 0, 1, 0, 1, 1])
kernel = NUTS(model)
mcmc = MCMC(kernel, num_warmup=500, num_samples=10000)
mcmc.run(jax.random.PRNGKey(0), data=data, n=len(data))
mcmc.print_summary()
posterior_samples = mcmc.get_samples()
```

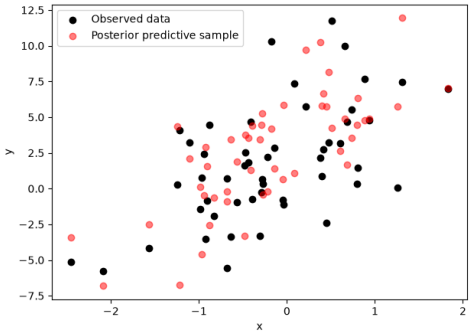
See notebook 1

A more complex model: Linear regression

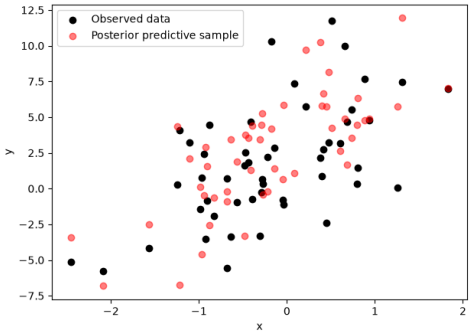
```
def model(x, y=None):
    intercept = numpyro.sample("intercept", dist.Normal(0, 10))
    slope = numpyro.sample("slope", dist.Normal(0, 10))
    sigma = numpyro.sample("sigma", dist.HalfNormal(5))
    mu = intercept + slope * x
    with numpyro.plate("data", len(x)):
        numpyro.sample("obs", dist.Normal(mu, sigma), obs=y)
```

See notebook 2

Did Our MCMC Sampler Work?



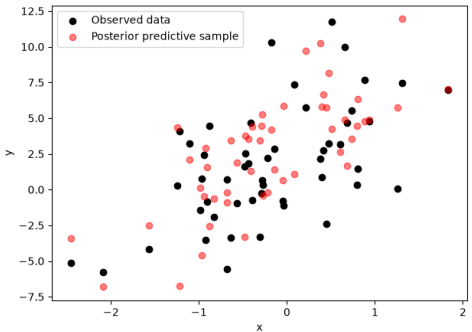
Did Our MCMC Sampler Work?



Important

Obtaining samples is not enough.

Did Our MCMC Sampler Work?



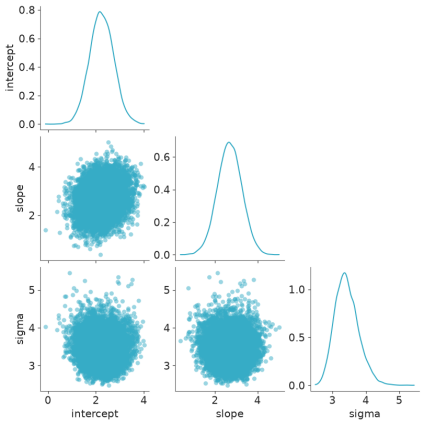
Investigate if output is meaningful:

- Local or global minimum? Look at sample convergence
- Efficient posterior exploration? Check effective sample size
- Enough samples? Compare uncertainty
- What can be wrong?

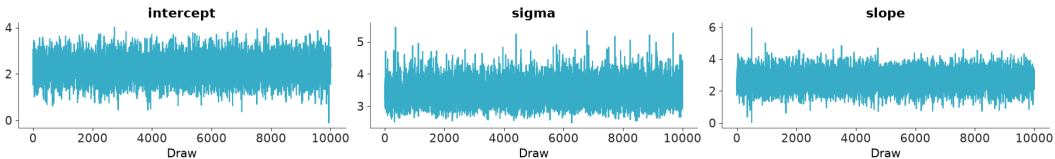
Important

Obtaining samples is not enough.

Inspect results: The triangle plot

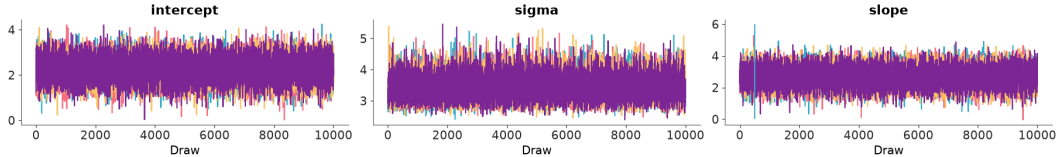


Visual Diagnostics: Trace Plots



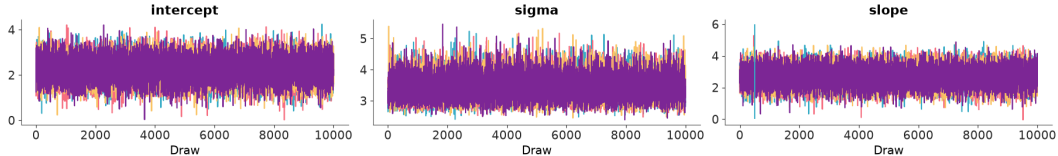
Visual Diagnostics: Trace Plots

Run several independent chains and plot the sampled values over time.



Visual Diagnostics: Trace Plots

Run several independent chains and plot the sampled values over time.



A healthy trace plot should look like:

- All chains explore the same region
- Chains overlap and mix well
- No long-term trends
- No chains getting stuck

Convergence: $\hat{R}$

$\hat{R}$ compares variation **between chains** with variation **within chains**.

Convergence: $\hat{R}$

$\hat{R}$ compares variation **between chains** with variation **within chains**.

$$\hat{R} \approx 1$$

means that all chains appear to sample from the same distribution.

Convergence: $\hat{R}$

$\hat{R}$ compares variation **between chains** with variation **within chains**.

$$\hat{R} \approx 1$$

means that all chains appear to sample from the same distribution.

Rule of thumb

$$\boxed{\hat{R} < 1.01}$$

is usually considered a good sign.

Convergence: $\hat{R}$

$\hat{R}$ compares variation **between chains** with variation **within chains**.

$$\hat{R} \approx 1$$

means that all chains appear to sample from the same distribution.

Rule of thumb

$$\boxed{\hat{R} < 1.01}$$

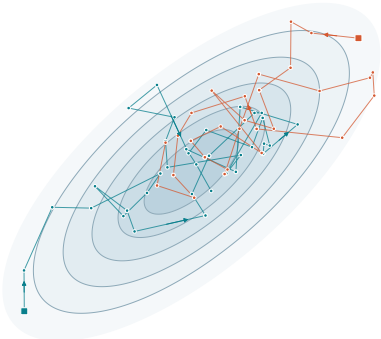
is usually considered a good sign.

If $\hat{R}$ is clearly larger than 1, the chains have probably not mixed sufficiently.

Effective Sample Size

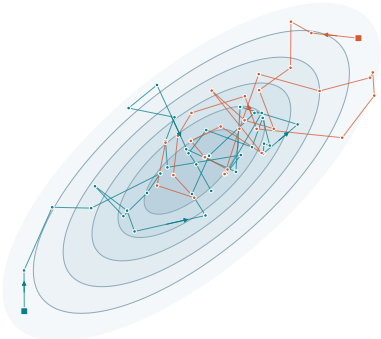
MCMC samples are correlated:

$N_{\text{independent}} < N_{\text{samples}}$



Effective Sample Size

MCMC samples are correlated:
$N_{\text{independent}} < N_{\text{samples}}$

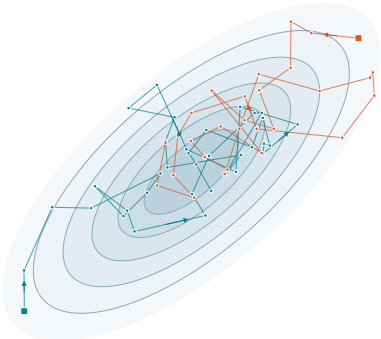


The **effective sample size (ESS)** estimates how many independent samples contain approximately the same information.

$$N_{\text{eff}} \leq N$$

Effective Sample Size

MCMC samples are correlated:
$N_{\text{independent}} < N_{\text{samples}}$



The **effective sample size (ESS)** estimates how many independent samples contain approximately the same information.

$$N_{\text{eff}} \leq N$$

Good sampling

Large ESS relative to the total number of samples means efficient exploration of the posterior.

How Many Samples Do We Need?

No universal number of MCMC samples.

How Many Samples Do We Need?

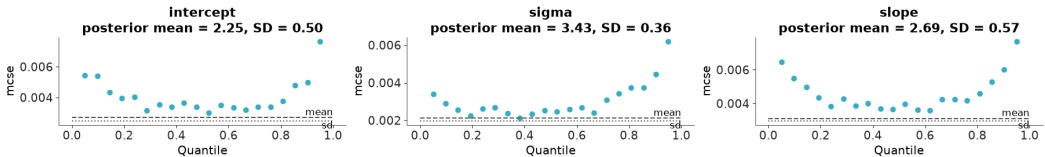
No universal number of MCMC samples.
Determines accuracy of the quantities we estimate.
For a posterior mean, the Monte Carlo Standard Error is approximately

$$\mathrm{MCSE} \approx \frac{\sigma}{\sqrt{N_{\mathrm{eff}}}},$$

How Many Samples Do We Need?

No universal number of MCMC samples.
Determines accuracy of the quantities we estimate.
For a posterior mean, the Monte Carlo Standard Error is approximately

$$\mathrm{MCSE} \approx \frac{\sigma}{\sqrt{N_{\mathrm{eff}}}},$$



How Many Samples Do We Need?

No universal number of MCMC samples.

How Many Samples Do We Need?

No universal number of MCMC samples.
Determines accuracy of the quantities we estimate.
For a posterior mean, the Monte Carlo Standard Error is approximately

$$\mathrm{MCSE} \approx \frac{\sigma}{\sqrt{N_{\mathrm{eff}}}},$$

How Many Samples Do We Need?

No universal number of MCMC samples.
Determines accuracy of the quantities we estimate.
For a posterior mean, the Monte Carlo Standard Error is approximately

$$\mathrm{MCSE} \approx \frac{\sigma}{\sqrt{N_{\mathrm{eff}}}},$$

In practice

Sample until the Monte Carlo error is small compared to the posterior uncertainty.

How Many Samples Do We Need?

No universal number of MCMC samples.
Determines accuracy of the quantities we estimate.
For a posterior mean, the Monte Carlo Standard Error is approximately

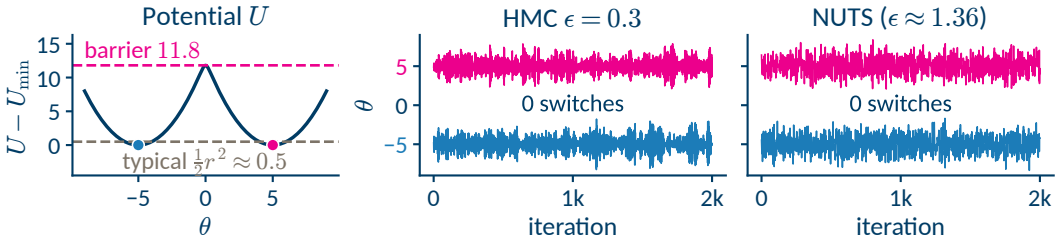
$$\mathrm{MCSE} \approx \frac{\sigma}{\sqrt{N_{\mathrm{eff}}}},$$

In practice

Sample until the Monte Carlo error is small compared to the posterior uncertainty.

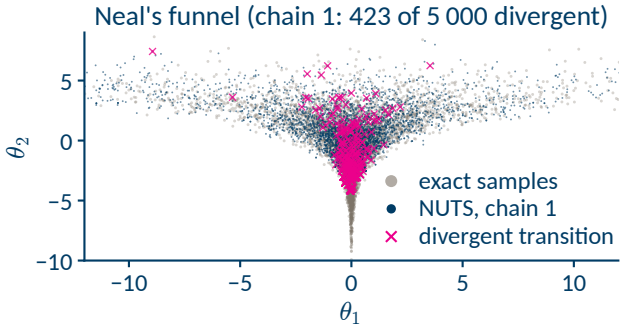
More iterations help only if the chain is actually mixing.

Limits of HMC/NUTS (1): Separate modes



- **Accurate HMC** ($\epsilon = 0.3$): Energy conservation requires $\frac{1}{2}r^2 \geq 11.8$ ($|r| > 4.9$) to cross $\Rightarrow$ no crossings.
- **NUTS** tunes ϵ, L to local curvature, not to a distant mode $\Rightarrow$ crossings unlikely
- Ensuring different initialisation of chains increase chance of discovery but will recover their relative probabilities. Truly separate modes requires special strategies.

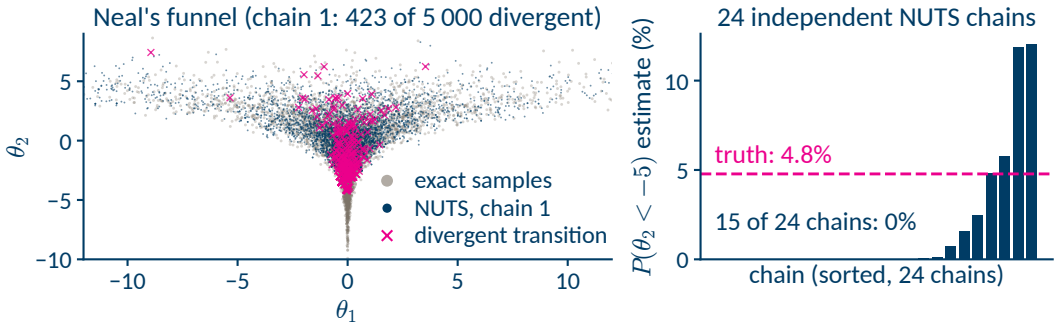
Limits of HMC/NUTS (2): Difficult geometry causes divergences



Neal's funnel[1]: Wide at the mouth (θ_2 large), narrow in the neck (θ_2 negative), so no single step size suits both.

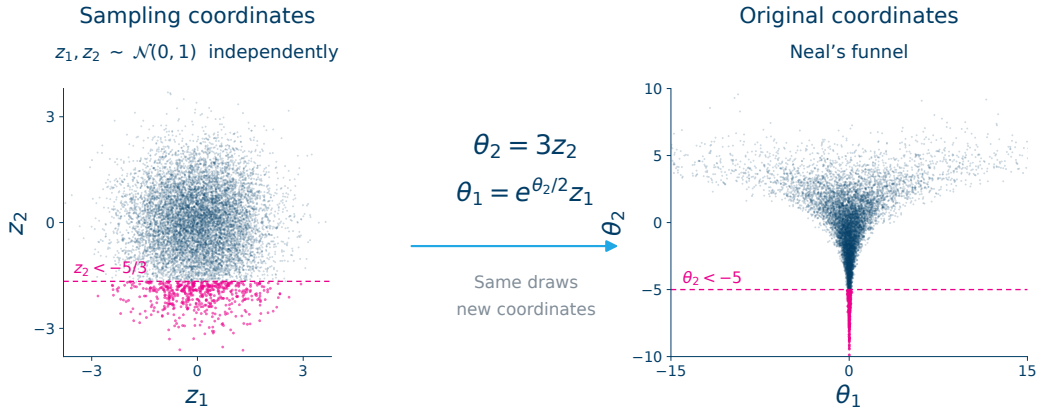
[1] $\theta = (\theta_1, \theta_2)$ with $\theta_2 \sim \mathcal{N}(0, 3^2)$ and $\theta_1 \mid \theta_2 \sim \mathcal{N}(0, e^{\theta_2})$

Limits of HMC (2): Difficult geometry causes divergences



Left: The NUTS chain hardly reaches the narrow/deep neck; a divergence is a trajectory whose energy error blows up (in 4–15% of the iterations of a chain, mostly in the neck).
Right: The estimate of $P(\theta_2 < -5)$ varies widely between chains.

Limits of HMC (2): Difficult geometry causes divergences



Magenta: the same tail event in both coordinate systems (true probability ≈ 4.8%).

10,000 independent normal draws, transformed exactly. Plot windows omit extreme draws; these are not NUTS output.

NUTS: What Can Go Wrong and What Helps?

Separated modes: Run chains from different initial values. Investigate identification and symmetries. Persistent separation may require tempering or another sampler.

Divergences: Inspect pair plots and parameter scales. Increase `target_accept_prob`. Reparameterization.

Funnel geometry: Try a non-centered parameterization. A smaller step size may be very inefficient.

Insufficient precision: If exploration is satisfactory, run longer. Check the MCSE of relevant quantities.

Before trusting results

Check divergences, agreement between chains, $\hat{R}$, bulk and tail ESS, and MCSE. More samples do not automatically fix poor exploration.

See notebook 2b

Beyond Linear Regression

Our linear regression already contained several unknown quantities:

$$\alpha, \quad \beta, \quad \sigma$$

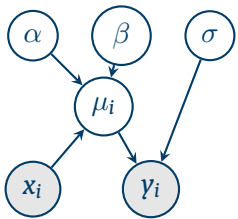
and observations y_i.

Beyond Linear Regression

Our linear regression already contained several unknown quantities:

$$\alpha, \quad \beta, \quad \sigma$$

and observations y_i.

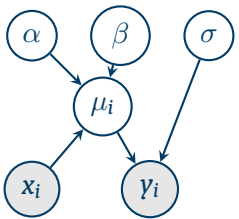


Beyond Linear Regression

Our linear regression already contained several unknown quantities:

$$\alpha, \quad \beta, \quad \sigma$$

and observations y_i.



This structure is not unique to regression models

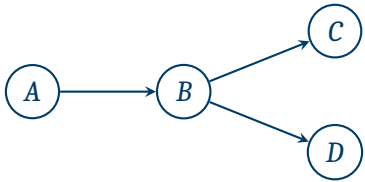
We can connect random variables in much more general ways.

Probabilistic Models as Graphs

A probabilistic model describes random variables and their conditional dependencies.

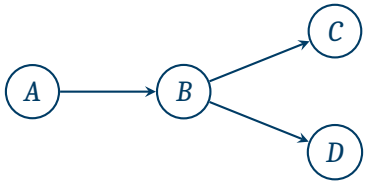
Probabilistic Models as Graphs

A probabilistic model describes random variables and their conditional dependencies.



Probabilistic Models as Graphs

A probabilistic model describes random variables and their conditional dependencies.

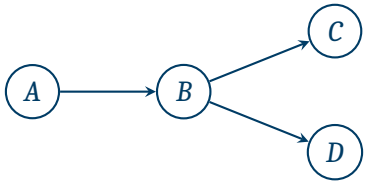


The graph defines a joint distribution:

$$p(A, B, C, D) = p(A)\, p(B \mid A)\, p(C \mid B)\, p(D \mid B).$$

Probabilistic Models as Graphs

A probabilistic model describes random variables and their conditional dependencies.



The graph defines a joint distribution:

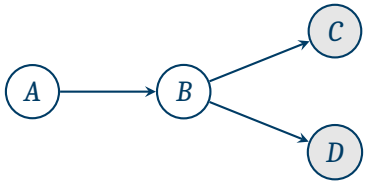
$$p(A, B, C, D) = p(A)\, p(B \mid A)\, p(C \mid B)\, p(D \mid B).$$

Directed Acyclic Graph — DAG

The graph tells us how the joint probability distribution factorises.

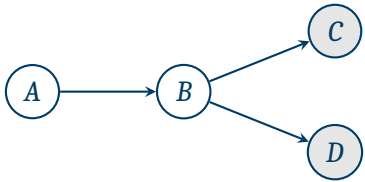
Observed and Latent Variables

A probabilistic model specifies both observed and latent variables.



Observed and Latent Variables

A probabilistic model specifies both observed and latent variables.

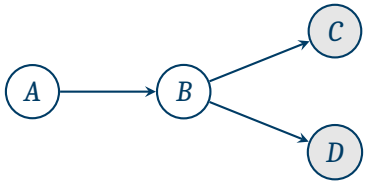


Suppose C and D are observed. Then inference gives us

$$p(A, B \mid C, D).$$

Observed and Latent Variables

A probabilistic model specifies both observed and latent variables.



Suppose C and D are observed. Then inference gives us

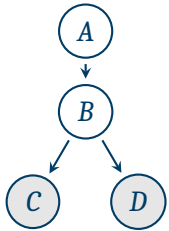
$$p(A, B \mid C, D).$$

General principle

Condition on what we know. Infer what we do not know.

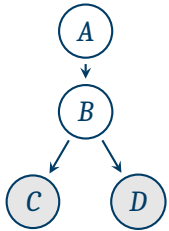
The Graph Becomes a NumPyro Model

```
def model(c=None, d=None):
    A = numpyro.sample(
        "A", dist.Normal(0, 1))
    B = numpyro.sample(
        "B", dist.Normal(A, 1))
    numpyro.sample(
        "C", dist.Normal(B, 1), obs=c)
    numpyro.sample(
        "D", dist.Normal(B, 1), obs=d)
```



The Graph Becomes a NumPyro Model

```
def model(c=None, d=None):
    A = numpyro.sample(
        "A", dist.Normal(0, 1))
    B = numpyro.sample(
        "B", dist.Normal(A, 1))
    numpyro.sample(
        "C", dist.Normal(B, 1), obs=c)
    numpyro.sample(
        "D", dist.Normal(B, 1), obs=d)
```



- `sample`: random variable in the model
- `obs=...`: condition on observed data
- Unobserved variables remain latent

One Model — Two Directions

The probabilistic model describes how variables are generated:

$$A \rightarrow B \rightarrow \begin{cases} C \\ D \end{cases}$$

One Model — Two Directions

The probabilistic model describes how variables are generated:

$$A \to B \to \begin{cases} C \\ D \end{cases}$$

Forward: Generate No observations:

$$A, B, C, D \sim p(A, B, C, D)$$

Simulate possible outcomes from the model.

Backward: Infer Observe C, D:

$$A, B \sim p(A, B \mid C, D)$$

Learn about unknown quantities.

One Model — Two Directions

The probabilistic model describes how variables are generated:

$$A \to B \to \begin{cases} C \\ D \end{cases}$$

Forward: Generate No observations:

$$A, B, C, D \sim p(A, B, C, D)$$

Simulate possible outcomes from the model.

Backward: Infer Observe C, D:

$$A, B \sim p(A, B \mid C, D)$$

Learn about unknown quantities.

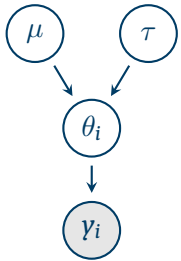
Probabilistic programming

The model stays the same. What changes is what we observe.

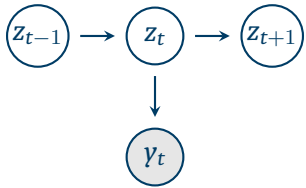
More Structure, Same Idea

The graph can become much more complex.

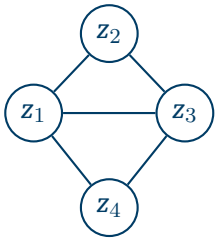
Hierarchical



Time series



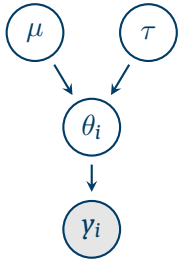
Networks



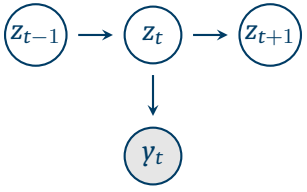
More Structure, Same Idea

The graph can become much more complex.

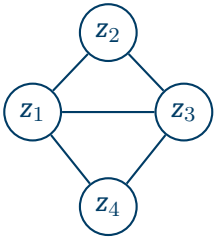
Hierarchical



Time series



Networks

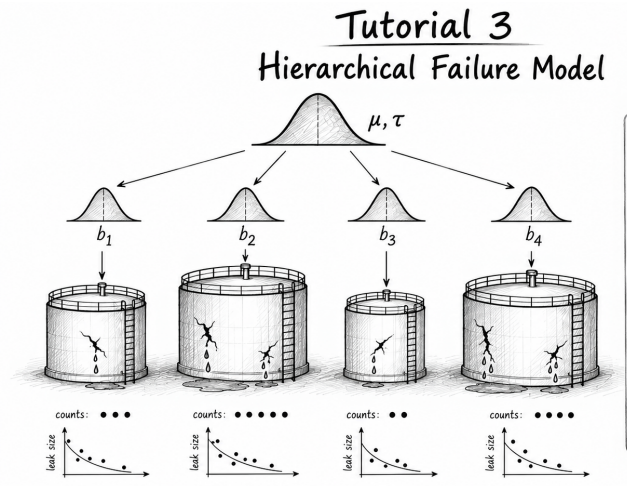


Same workflow

Model dependencies → add observations → infer latent quantities → generate predictions.

ai·D Norwegian Centre on AI for Decisions

Tutorial 3: Hierarchical Failure Model



$$b_i \sim \mathcal{N}(0, \tau)$$

$$\beta_i = \exp(b_i)$$

$$\lambda_i = \sigma \left(\frac{V_i}{V_0}\right)^{\alpha} \beta_i$$

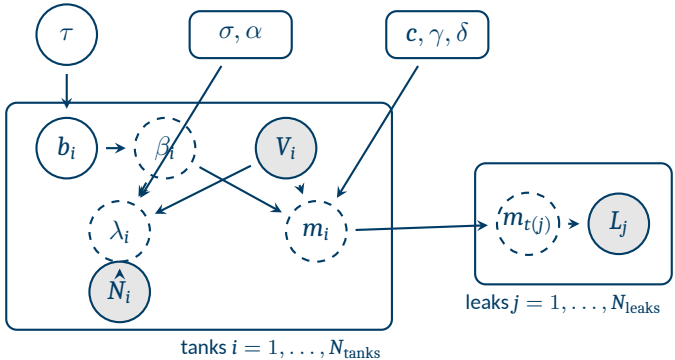
$$\text{counts}_i \sim \text{Poisson}(\lambda_i)$$

$$m_i = c \left(\frac{V_i}{V_0}\right)^{\gamma} \beta_i^{\delta}$$

$$\text{leak_size}_{ij} \sim \text{Exponential}\left(\frac{1}{m_i}\right)$$

Tutorial 3: Hierarchical Failure Model

Each tank has its own latent risk factor, while global parameters are shared across all tanks.



white = latent, grey = observed, dashed = deterministic $\beta_i = \exp(b_i)$ is shared by both observation models.

Hierarchical Failure Model: Generative Story

For tank i, normalise volume as $v_i = V_i/100$.

Tank effects

$$b_i \sim \mathcal{N}(0, \tau), \qquad \beta_i = \exp(b_i).$$

Failure counts

$$\lambda_i = \sigma v_i^{\alpha} \beta_i, \qquad N_i \sim \text{Poisson}(\lambda_i).$$

Leak sizes

$$m_i = c v_i^{\gamma} \beta_i^{\delta}.$$

For leak j from tank $t(j)$:

$$L_j \sim \text{Exponential}\left(\text{mean} = m_{t(j)}\right).$$

Partial pooling

Each tank has its own β_i, but all β_i are tied together through the shared scale τ.

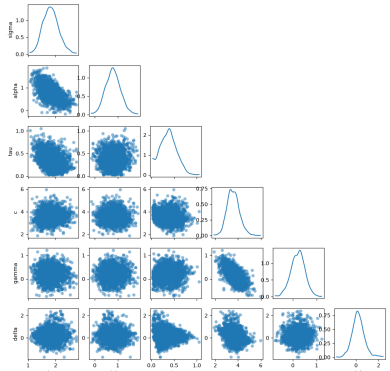
See notebook 3

Hierarchical Failure Model: What the Posterior Learns

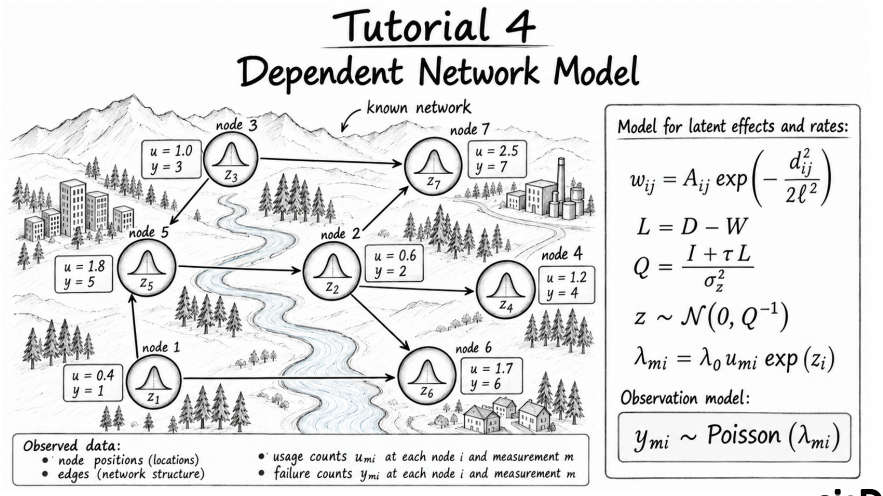
The posterior is not just a set of independent parameter estimates.

- Global behaviour: $\sigma, \alpha, \mathbf{c}, \gamma, \delta$
- Between-tank variation: τ
- Tank-specific latent risks: $\beta_1, \ldots, \beta_N$
- Posterior correlations between parameters

Interpretation: estimates for tanks with few observations are shrunk toward the population mean; tanks with strong evidence can deviate.

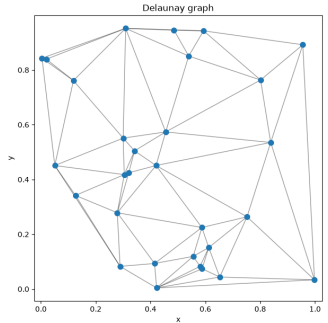


Tutorial 4: Dependent Network Model

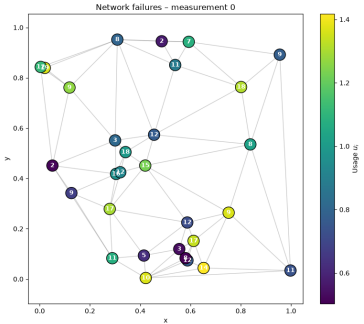


Tutorial 4: Dependent Network Model

The known network structure tells us which node effects should be related.



known graph

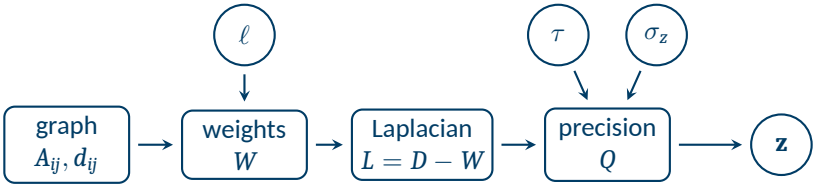


observed failures for one measurement

The prior encourages connected nodes to have similar latent effects z_i.

Network Model: From Graph to Latent Field

The known graph is converted into a correlated multivariate prior for the latent vector $\mathbf{z}$.



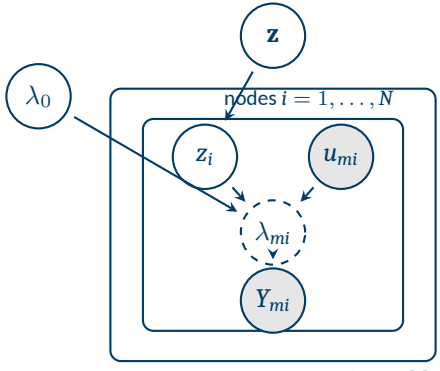
$$w_{ij} = A_{ij} \exp\left(-\frac{d_{ij}^2}{2\ell^2}\right), \qquad Q = \frac{I + \tau L}{\sigma_z^2}, \qquad \mathbf{z} \sim \mathcal{N}(0, Q^{-1}).$$

Important: $\mathbf{z}$ is sampled jointly as one correlated vector, not as independent node-wise samples.

Network Model: Observation Model with Plates

For measurement m and node i,

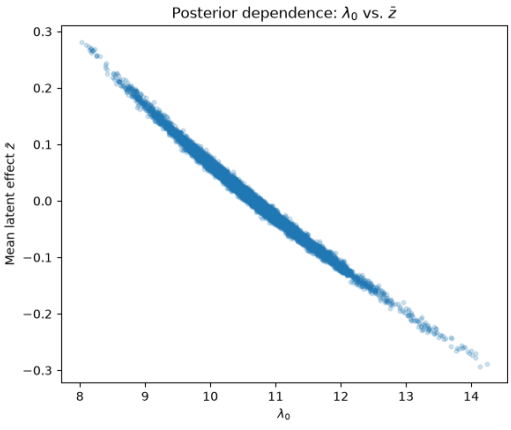
$$\lambda_{mi} = \lambda_0 \, u_{mi} \exp(z_i), \qquad Y_{mi} \sim \text{Poisson}(\lambda_{mi}).$$



See notebook 4

Network Model: What to Look For

The model can reveal dependencies between global parameters and latent node effects.



The observation rate is generated as

$$\lambda_{mi} = \lambda_0 \, u_{mi} \exp(z_i),$$

or equivalently

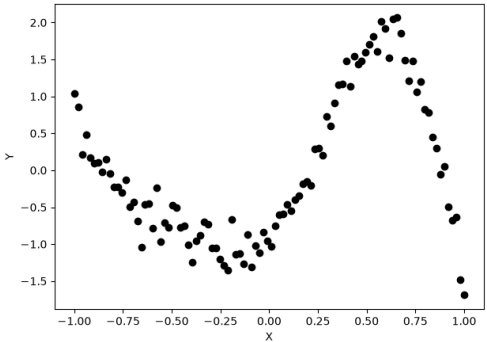
$$\log \lambda_{mi} = \log \lambda_0 + \log u_{mi} + z_i.$$

- λ_0 controls the global failure level.
- z_i shifts the rate at individual nodes.
- For a given fitted rate, a larger λ_0 can be offset by smaller values of z_i.

ai·D Norwegian Centre on AI for Decisions

Tutorial 5: Bayesian Neural Network

A neural network becomes Bayesian by placing priors on its weights.

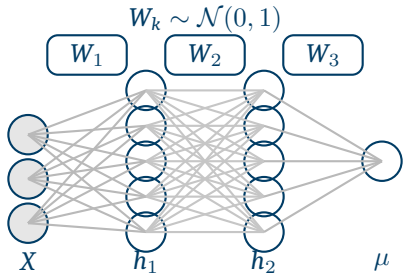


- Nonlinear relationship
- Limited observations
- Quantify uncertainty in regions with sparse observations

Key idea: the weights are uncertain, so the fitted function and predictions are uncertain.

Bayesian Neural Network: Deterministic Forward Pass

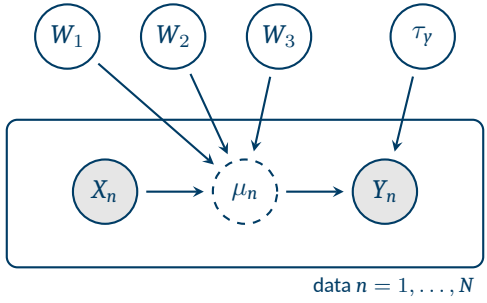
The network computation is deterministic once the weights are fixed.



$$h_1 = \tanh(XW_1), \qquad h_2 = \tanh(h_1 W_2), \qquad \mu = h_2 W_3.$$

Bayesian Neural Network: Probabilistic Model

The weights are global latent variables; observations are repeated over data points.



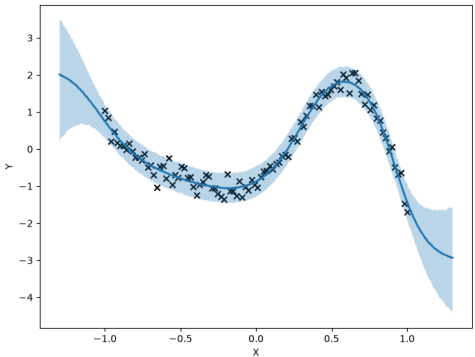
$$W_1, W_2, W_3 \sim \mathcal{N}(0, 1), \qquad \tau_Y \sim \text{Gamma}(3, 1), \qquad Y_n \sim \mathcal{N}(\mu_n, \tau_Y^{-1/2}).$$

Inference target: $p(W_1, W_2, W_3, \tau_Y \mid X, Y).$

See notebook 5

Bayesian Neural Network: Posterior Predictive

For each posterior sample of the weights, evaluate the network to obtain a possible function.



- Dark curve: posterior predictive mean
- Band: predictive uncertainty
- Predictive uncertainty may increase in regions with sparse observations

Main lesson: a Bayesian neural network gives a distribution over functions, not just one fitted curve.